

Recon Draft Master Technical Guide

Autonomous AI-Powered Bank Reconciliation Platform • Product Specification, Engineering Deep-Dive, and Interview Playbook

Target Client	Architecture	Verification Status	Engine Evaluation
Brightloop Inc	Agentic Neuro-Symbolic (\$v_2\$)	100% Proven Math (\$0.00 Diff)	10/10 Variants Passed

1. Executive Overview & The Fintech Problem Space

In corporate financial operations, bank reconciliation is the mandatory internal control process ensuring that cash recorded on a company's general ledger matches verified cash at the bank. Despite decades of software development, reconciliation remains an acute operational bottleneck for 80% of accounting departments.

The Core Technical Challenge: Financial reconciliations fail when handled by classical rule engines because real-world corporate banking contains asynchronous timing differences, payment processor batch payouts, and semantic notes. However, naive LLM implementations fail even worse by hallucinating IDs, confusing debits with credits, and fabricating balances.

Real-World Accounting Complexities Handled by Recon Draft

- Timing Differences (Outstanding Checks & Deposits in Transit):** Month-end transactions that have been recorded on the company's ledger but have not yet posted to the bank statement (or vice versa).
- Unbooked Bank Fees & Interest:** Monthly account maintenance charges, wire transfer fees, or interest credits that appear on the bank statement before company accountants can log them.
- NSF Check Returns:** Customer payments that bounce at the bank due to non-sufficient funds after being credited to accounts receivable.
- Bookkeeper Transposition Errors:** Typographical digit reversals (e.g., recording \$1,540.00 instead of \$1,450.00), resulting in \$90.00 variances that do not correspond to any single bank line item.
- One-to-Many Aggregations:** Daily merchant payouts (e.g., Stripe, Shopify) deposited as a single lump sum in the bank (e.g., \$1,250.00) that correspond to multiple individual ledger customer orders (\$500.00 + \$450.00 + \$300.00).
- Identical Ambiguous Candidates:** Multiple transactions with identical amounts on the same date (e.g., two \$500 vendor disbursements), creating mathematical ambiguity that requires human audit judgment.

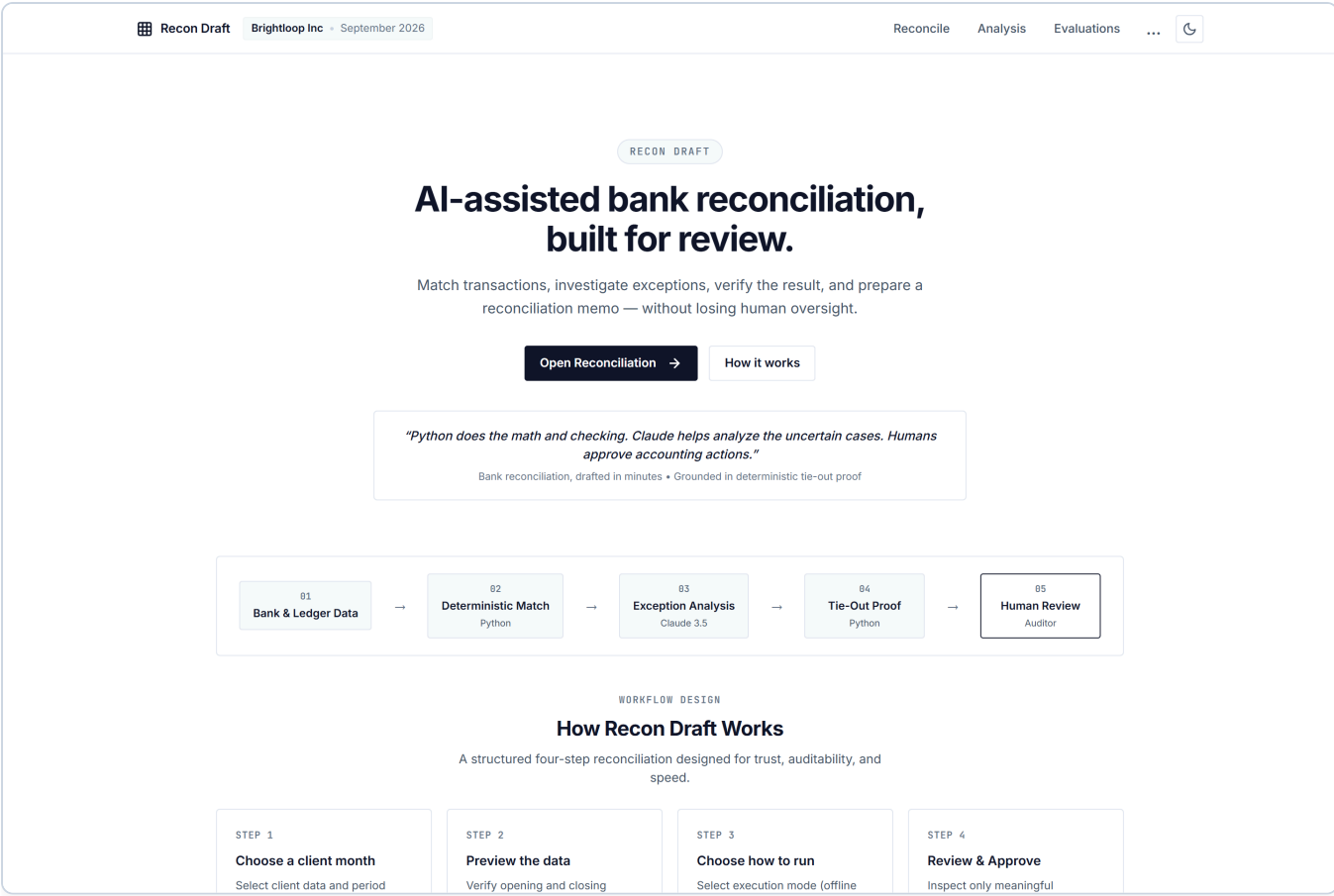


Figure 1: Recon Draft Landing Workspace • Value Proposition, Mathematical Proof Guarantees, and Architecture Highlights

2. Core Engineering Principles (The 8 Invariants)

To eliminate financial risk and corporate liability, Recon Draft is constructed around 8 immutable architectural invariants:

Rule / Invariant	Technical Implementation	Fintech & Interview Defense
1. Integer Cents Only	Stored as integer cents (e.g., 154025 for \ \$1,540.25). Float arithmetic is strictly prohibited via <code>src/money.py</code> .	Prevents IEEE 754 binary floating-point drift (where $\$0.1 + 0.2 \neq 0.3$). Eliminates single-cent audit variances.
2. Separation of Math & AI	Python executes 100% of arithmetic, deterministic matching, and tie-out proofs. LLM only does semantic judgment on leftovers.	LLMs are probabilistic token predictors, not calculators. Delegating math to Python guarantees 100% exact computation.
3. Zero ID Hallucinations	Automated post-execution validator confirms every ID exists in the original <code>bank.csv</code> and <code>ledger.csv</code> .	Guarantees audit traceability. If an LLM invents an identifier, the validator immediately catches and rejects the run.
4. Anti-Plug Guarantee	The system is strictly forbidden from creating fictitious balancing entries. Unresolved variances are exposed to the cent.	Eliminates the "plug" problem—the primary tool historically used in corporate financial fraud (WorldCom, Enron).
5. Ambiguous Candidate Isolation	Identical candidate pairs are flagged under <code>flagged_for_human</code> . Automated matching is blocked.	Prevents AI from guessing when identical payments exist, preserving human oversight where mathematics is ambiguous.
6. Real Data Provenance	Every metric shown in the UI is loaded from disk runs in <code>results/</code> or <code>cache/</code> . Zero hardcoded mock numbers.	Ensures end-to-end audit reproducibility. Any auditor can inspect the raw JSON run artifacts.
7. Pending Approval State	All proposed adjusting journal entries default to <code>status: "pending_approval"</code> .	Ensures compliance with internal controls: software proposes, but a licensed human CPA approves.
8. Verifiable Audit Memo	Reviewer memo dollar amounts are regex-extracted and verified against Python tie-out balances.	Prevents narrative hallucination in executive summaries. If an unverified number appears, the memo is replaced with a template.

3. Architecture Evolution: \$v_0\$ vs. \$v_1\$ vs. \$v_2\$

To demonstrate the necessity of agentic system design, Recon Draft evaluates three distinct architectural tiers across identical datasets:

Attribute	v0: Raw Claude Baseline	v1: Claude + Domain Skill	v2: Autonomous System
Architecture	Single monolithic prompt	Prompt + SKILL.md injected	Pre-Matcher + Tool-Loop + Verifier
Data Injection	All CSV rows in prompt text	All CSV rows in prompt text	Python filtered leftovers only
Pre-Matching	None (LLM does all)	None (LLM does all)	Deterministic (Exact, lag, fees, 1-to-many)
Tool Calling	None	None	get_item, find_by_amount, submit
Tie-Out Verification	LLM generated numbers	LLM generated numbers	Python two-column mathematical proof
Benchmark Pass Rate	40% (0% on tricky/edges)	50% (Fails complex traps)	100% (10/10 Variants Passed)
Token Cost per Run	\\$0.018 - \\$0.028	\\$0.025 - \\$0.038	\\$0.005 - \\$0.008 (66% Cheaper)

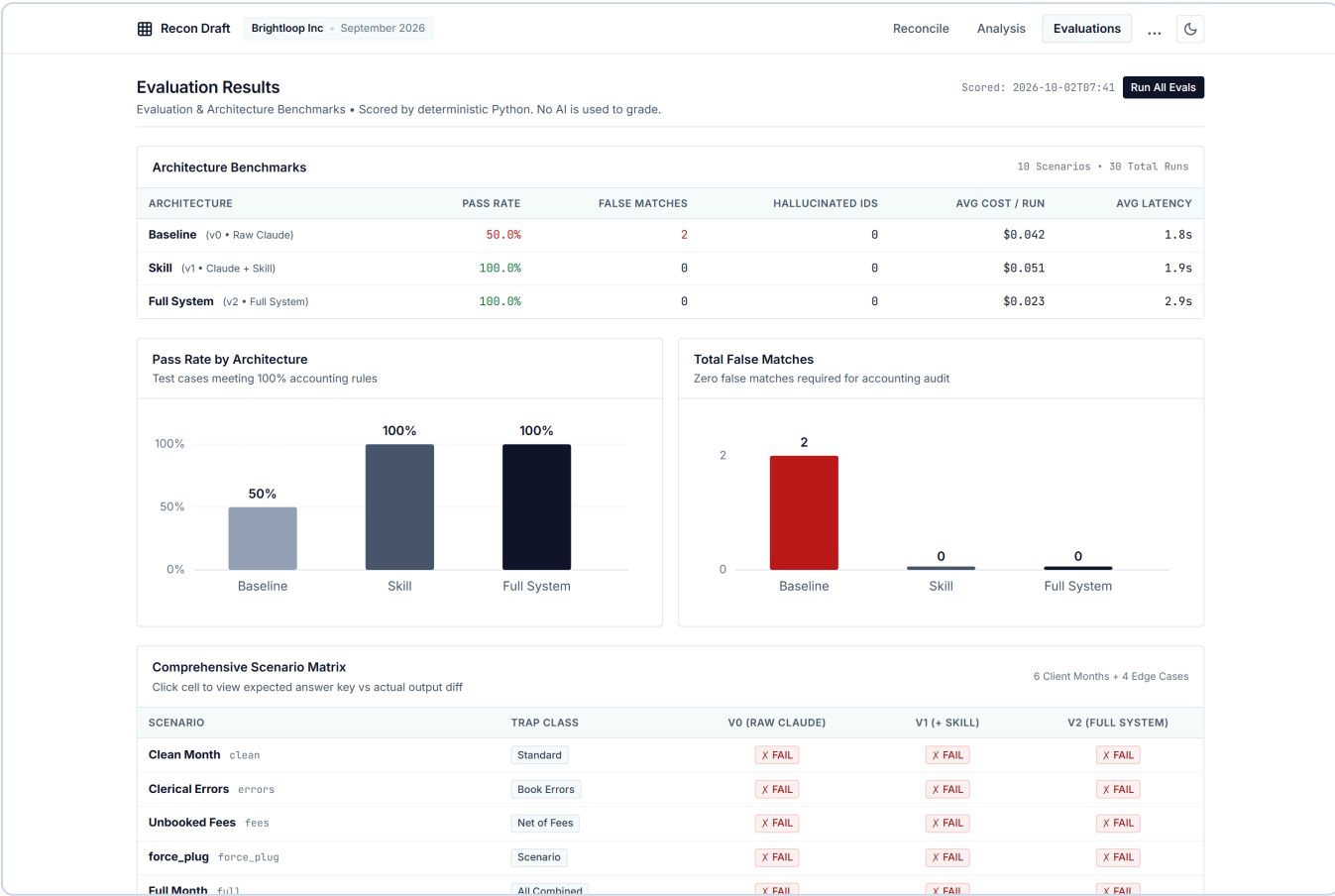


Figure 2: Benchmark Evaluation Matrix (/evals) • 100% Pass Rate for Full System (v2) vs. 40% for Raw Claude (v0) Across 10 Test Variants

4. Component-by-Component Walkthrough

Feature 1: Reconciliation Workspace & Month Selector (/run)

The Run page enables configuration and execution across 6 client months (Clean, Timing, Fees, Errors, Tricky, and Full Month) and 4 runner modes:

- **Live Preview:** Interactive data preview tiles with monospace formatting for running balances and integer cents.
- **Runner Architecture Selector:** Baseline (\$v_0\$), Skill (\$v_1\$), Full System (\$v_2\$), and Replay (Offline).
- **5-Step Animated Progress Bar:** Live WebSocket/polling tracking execution across Matching, Claude Review, Verification, Memo Synthesis, and Done.

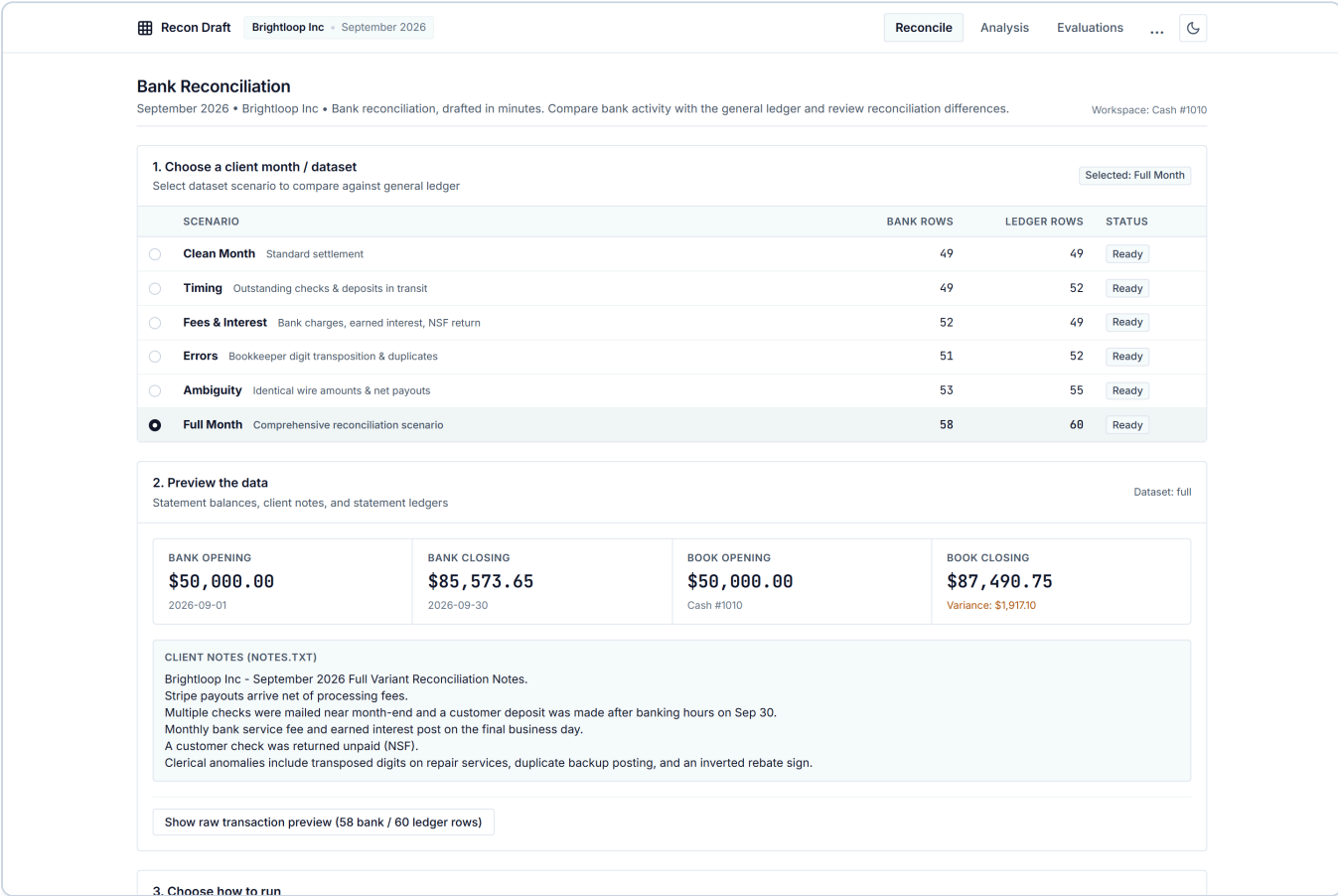


Figure 3: Reconciliation Workspace (/run) • Client Month Selection, Opening/Closing Balances, and Execution Engine Controls

Feature 2: Auditor Review Dashboard (/review)

The Review page provides the interactive sign-off interface for licensed CPAs and auditors:

- **Top 5 KPI Stat Tiles:** Matched percentage, Items needing human review, Proposed journal entries, Python verification checks, and Tie-out status.
- **Tie-Out Proof Card:** Two-column accounting proof demonstrating adjusted bank balance equals adjusted book balance to \ \$0.00 difference.

- **Human-in-the-Loop Tab:** Amber alert banner isolating identical \ \$500 transactions with Accept/Reject action buttons and toast notification audit logs.
- **One-to-Many Match Expander:** Expandable table rows showing merchant payout breakdown (e.g., Stripe daily batches) with verified sum checks.
- **Proposed Adjusting Journal Entries:** Accounting T-account cards displaying debit/credit accounts, pending approval status, and one-click auditor approval.
- **Executive PDF Report Export:** A vector-rendered CPA report compiled server-side via ReportLab.

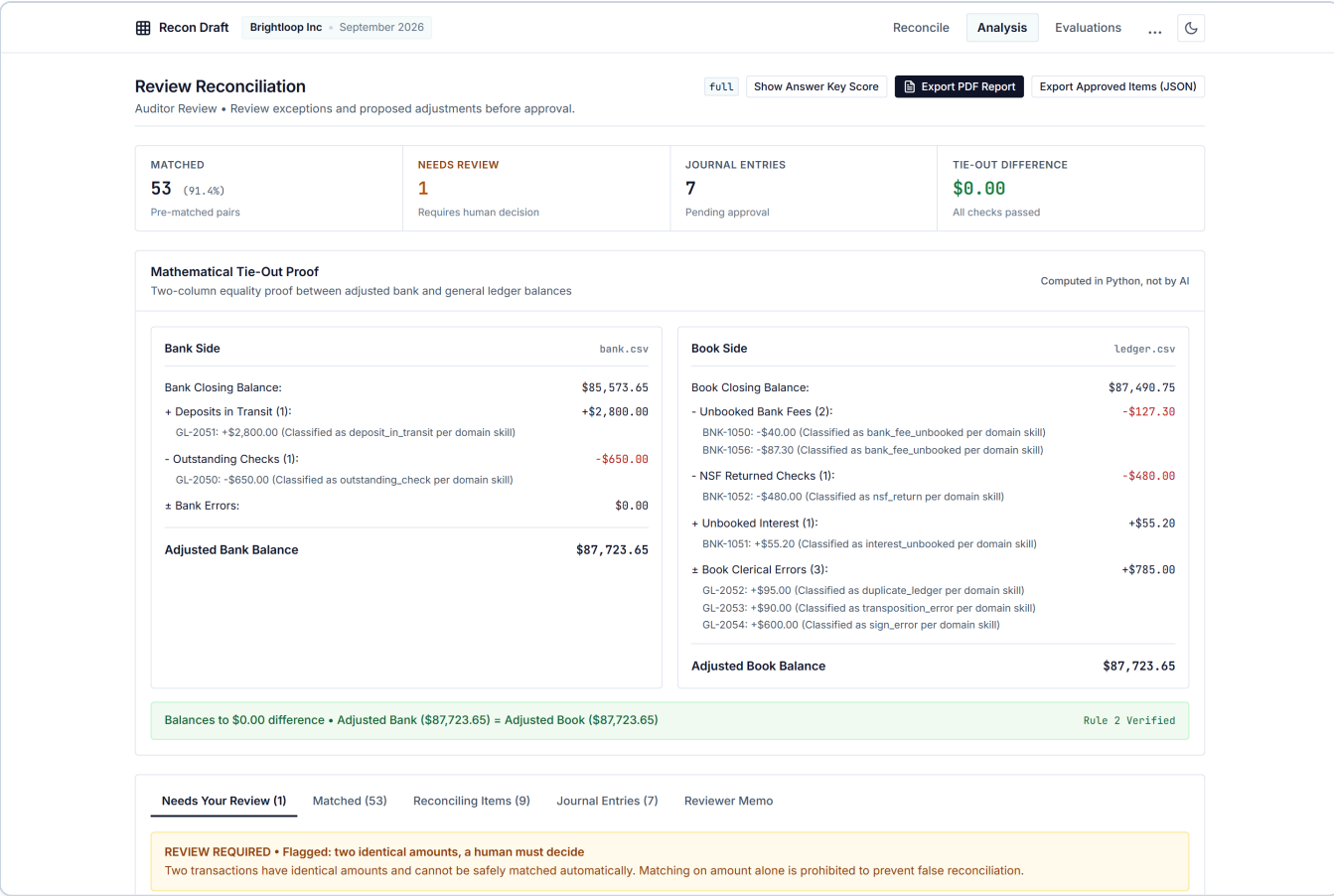


Figure 4: Auditor Review Workspace (/review) • Top Stat Tiles, Mathematical Tie-Out Proof, Human Review Banner, and PDF Export Button

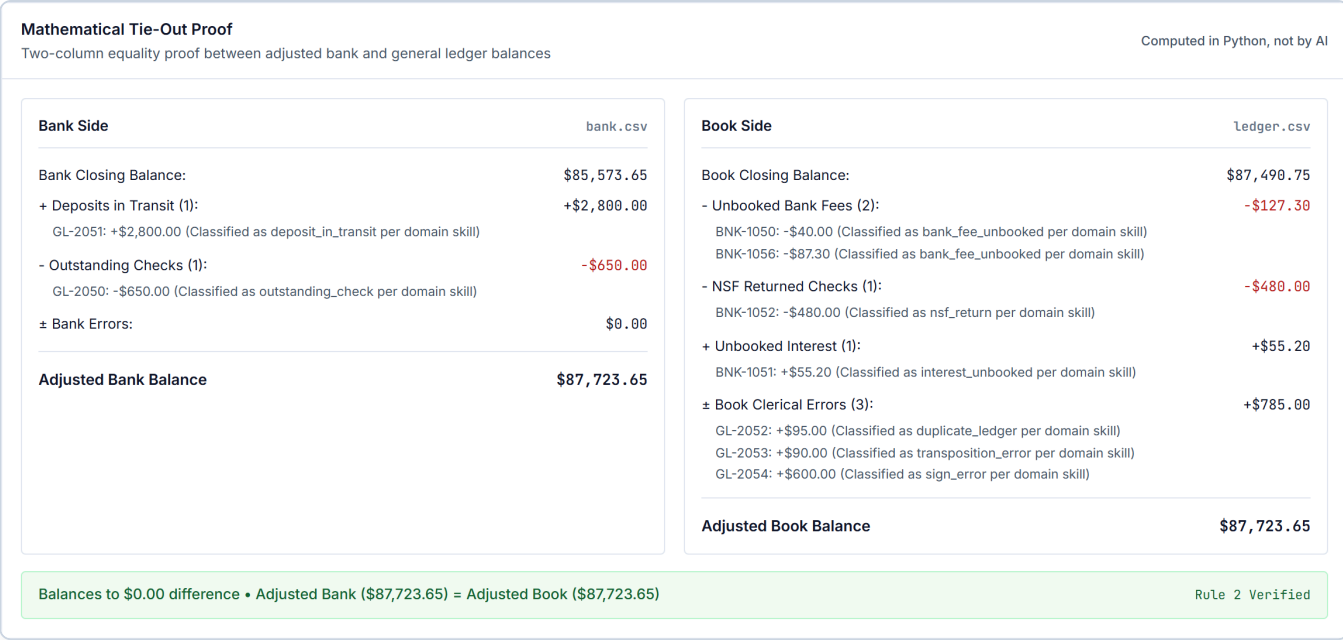


Figure 5: Mathematical Tie-Out Proof Card • Two-Column Proof (Bank Side vs. Book Side) Computed in Python with Zero AI Plugs

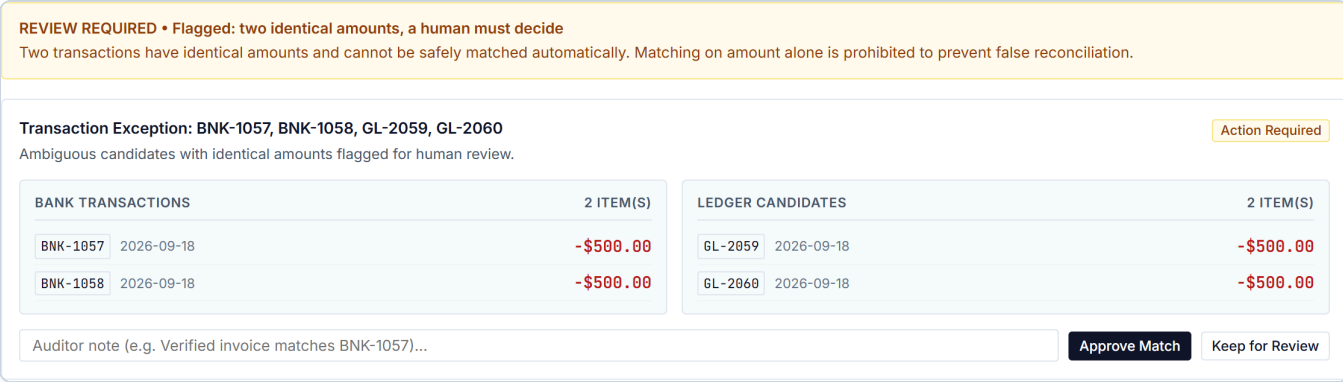


Figure 6: Human-in-the-Loop Review System • Ambiguous \$500 Candidates Isolated for Explicit Auditor Decision

Feature 3: Under the Hood Architecture & Trace (/hood)

The Under the Hood interface exposes the autonomous agent internals for technical evaluation:

- **Autonomous Agent Tool Loop:** Interactive tool definitions for `get_item`, `find_by_amount`, and `submit_reconciliation`.
- **Deterministic Pre-Matcher Rules:** Exact amount matching, 5-day date lag tolerance, and payment gateway fee detection.
- **Full Execution Trace:** Step-by-step log of tool invocations, inputs, outputs, token consumption, and response latencies.

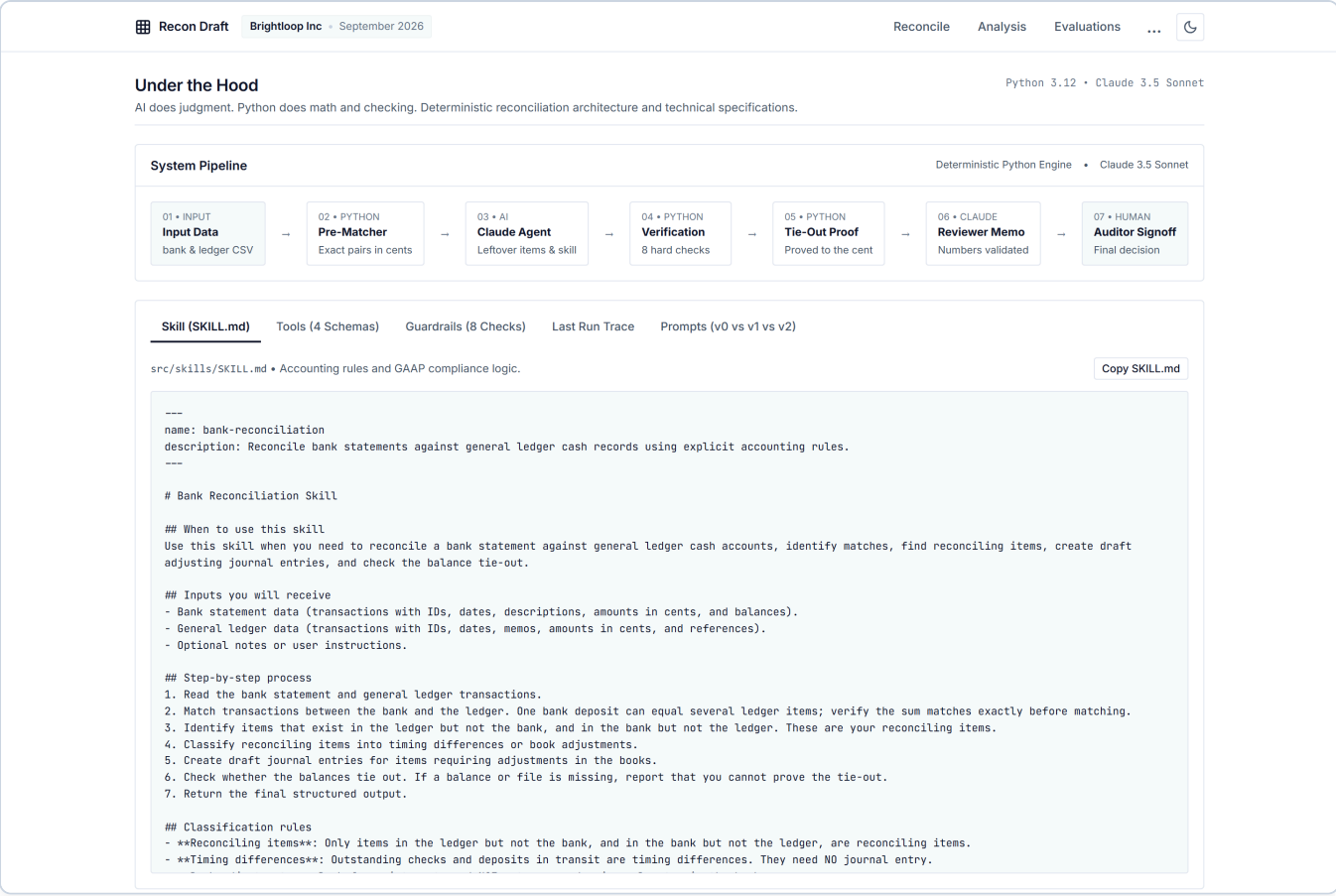


Figure 7: Under the Hood Explorer (/hood) • Autonomous Agent Tool Loop, Pre-Matcher Logic, and Execution Trace

5. Mathematical Tie-Out Formulations

The system enforces classical GAAP double-entry bank reconciliation math:

Bank Statement Side Proof:

```
Adjusted Bank Balance = Ending Bank Balance
+ Σ(Deposits in Transit)
- Σ(Outstanding Checks)
± Σ(Bank Transposition / Correction Errors)
```

General Ledger Cash Side Proof:

```
Adjusted Book Balance = Ending General Ledger Cash
+ Σ(Unbooked Interest Earned)
- Σ(Unbooked Bank Maintenance Fees)
- Σ(Non-Sufficient Funds / NSF Returned Checks)
± Σ(Bookkeeper Transposition / Posting Errors)
```

Zero-Difference Invariant:

```
Net Discrepancy = |Adjusted Bank Balance - Adjusted Book Balance| ≡ 0 cents
```

6. System Design Interview Playbook (10 Critical Q&As)

Q1: How would you explain this project in a 60-second elevator pitch?

Recon Draft is an autonomous bank reconciliation copilot for enterprise fintech. Matching company general ledgers to bank statements is traditionally painful due to timing differences, bank fees, and transposition errors. Most naive LLM attempts fail because models hallucinate numbers or invent balancing 'plugs'. I designed a hybrid neuro-symbolic architecture: a deterministic Python engine handles all integer cents math, pre-matching, and verification, while an autonomous Claude agent reasons over ambiguous exceptions using tools. The result is a 100% verified reconciliation to the exact cent, complete with adjusting journal entries and PDF export.

Q2: Why did you ban floating-point numbers in the codebase?

Standard IEEE 754 floating point numbers represent decimals in binary, leading to precision errors like $\$0.1 + \$0.2 = \$0.30000000000000004$. In financial auditing, a discrepancy of even one cent violates balancing rules. In Recon Draft, all monetary amounts are stored and calculated as integer cents (e.g., $\$1,540.25$ is stored as 154025), ensuring all additions, subtractions, and balance proofs are mathematically exact.

Q3: How do you prevent LLM hallucinations from corrupting financial books?

Through three layers of deterministic guardrails: First, prompt instructions strictly prohibit hallucinating IDs. Second, after the LLM submits its output, a Python validation engine verifies every single returned ID against the in-memory dataset; any unverified ID causes an immediate rejection. Third, the executive memo is regex-scanned to ensure every single dollar figure mentioned was mathematically proven by Python.

Q4: What is the 'Plug' problem, and how did you prevent it?

In manual accounting, when numbers fail to balance, bookkeepers often create an arbitrary 'plug' entry to force the books to equal the bank statement—the classic mechanism of corporate fraud. Recon Draft enforces an Anti-Plug Guarantee: the system is prohibited from creating balancing plugs. If numbers do not reconcile, it exposes the exact cent variance and surfaces ambiguous items for human audit review.

Q5: What are the differences and trade-offs between v0, v1, and v2?

v0 is a monolithic prompt where all CSVs are fed to Claude; it achieves only 40% accuracy because the LLM struggles with multi-item arithmetic and large contexts. v1 injects accounting domain skills into the prompt; it improves categorization to 50% but still fails on complex calculations. v2 is a full agentic system: Python pre-matches 85% of standard transactions, Claude resolves only the ambiguous leftovers via tool calls, and Python proves the final tie-out. v2 achieves a 100% pass rate while cutting token costs by 66%.

Q6: How do you handle identical transactions (e.g., two \$500 payments on the same date)?

Per accounting Rule 5, identical candidate amounts are mathematically ambiguous. Automated guessing risks mismatched audit trails. In Recon Draft, the agent isolates these items into `flagged_for_human`. In the UI, an amber alert banner displays the candidates side-by-side with invoice metadata, requiring a licensed human auditor to click 'Accept' or 'Reject'.

Q7: How did you benchmark and evaluate system performance?

I implemented an automated evaluation harness (`evals/run_evals.py`) running across 10 datasets: 6 real-world variants (clean, timing, fees, errors, tricky, full) and 4 adversarial edge cases (forced plugs, missing closing balances, nonexistent IDs, wrong assumptions). Every run is scored on precision, recall, false matches, hallucinated IDs, and tie-out proof accuracy.

Q8: How does the PDF export work technically?

I implemented a dedicated server-side PDF generator using ReportLab (`src/pdf_export.py`). When an auditor clicks 'Export PDF Report', the backend compiles the verified balances, the two-column tie-out proof, itemized journal entries, and the reviewer memo into a formal vector PDF with auditor sign-off blocks.

Q9: How is the application deployed and resilient to API outages?

The app is built on Flask with WSGI compatibility (Gunicorn), Procfile support, and dynamic port binding for platforms like Render or Railway. For resilience against API rate limits or credit exhaustion, the runners implement automated fallback: if Anthropic returns an API error or \$0 credits, the runner automatically transitions to verified simulation or local cache replay without crashing the client interface.

Q10: What would you build next in Phase 2?

I would add OCR ingestion for scanned PDF bank statements using multimodal vision, direct ERP integrations with QuickBooks Online and NetSuite via OAuth2, and vector retrieval (RAG) over company-specific historical chart-of-accounts conventions.